Project Coordinator & Guide

Dr. Leena Silvoster

Associate Professor

College of Engineering, Karunagappally

01. Overall End-to-End System Flow (Figure 4.3)

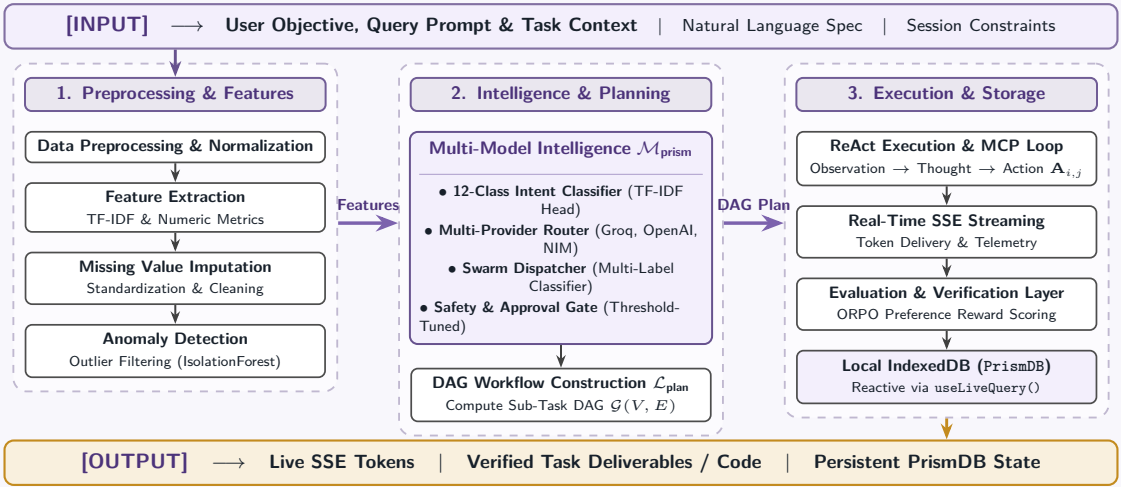
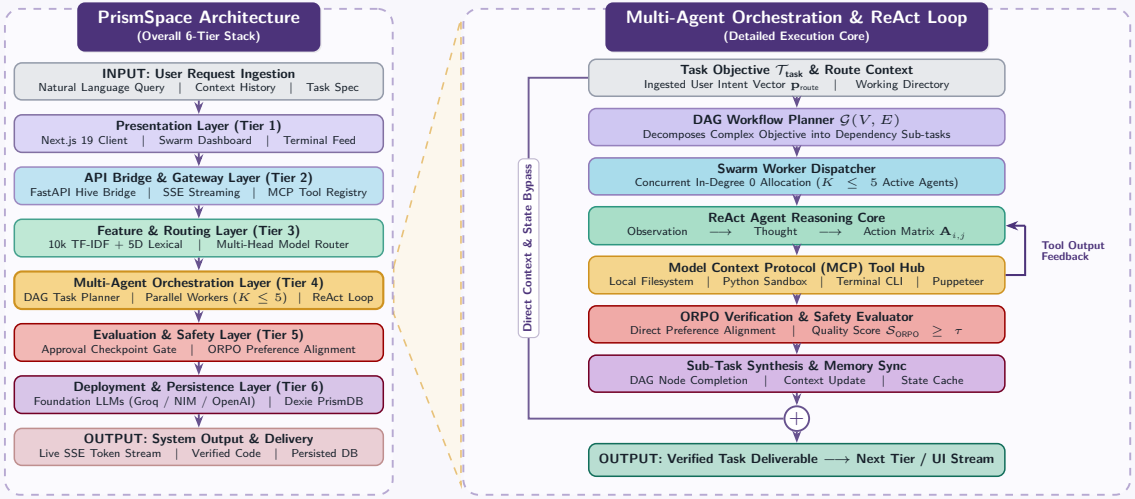


Figure 4.3: End-to-End System Flow Architecture (PrismSpace)

02. System Architecture: 6-Tier Stack & Multi-Agent Workflow



Glossary: **DAG** = dependency graph | **ReAct** = observe–reason–act | **MCP** = tool protocol | **ORPO** = preference evaluator | **SSE** = live event stream

03. Browser Database Schema: PrismDB (Figure 4.4)

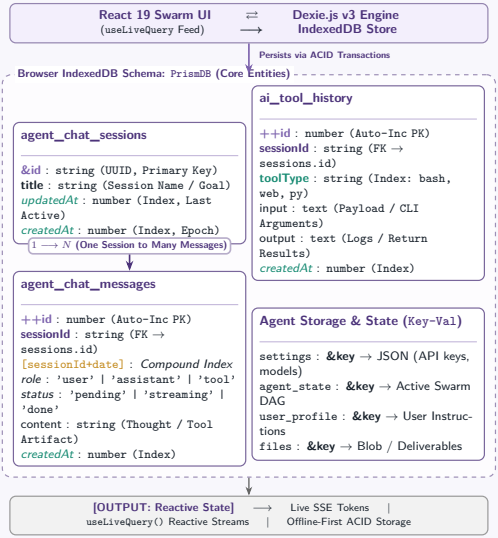


Figure 4.4: Browser IndexedDB Database Schema and Reactive Data Architecture (PrismDB)

Glossary

1. PrismDB
Client-side IndexedDB storing AI swarm sessions, chat & tool telemetry.

2. Dexie.js v3
TypeScript IDB wrapper handling schema migrations & ACID transactions.

3. PK & FK Keys
`&id` (UUID PK), `++id` (Auto-Inc PK), and $1 \rightarrow N$ relational links to sessions.

4. Compound Idx
B-tree index `[session+date]` for instant sub-millisecond query slicing.

5. `useLiveQuery()`
Reactive hook auto-rerendering React UI components on local DB changes.

Algorithm

Step 1: Data collection and Preprocessing

1. Import and unify benchmark records ($\mathcal{D}$), mapping heterogeneous prompt schemas into a structured format.
2. Clean the dataset (strip malformed Unicode, redundant whitespace, and duplicate prompt-response pairs).
3. Standardize continuous features (tokens, complexity, latency) using z-score normalization: $z = (x - \mu)/\sigma$.
4. Split the dataset into:
   - 80% Training set ($\mathcal{D}_{\text{train}}$)
   - 10% Validation set ($\mathcal{D}_{\text{val}}$)
   - 10% Testing set ($\mathcal{D}_{\text{test}}$)

Step 2: Swarm Graph Construction

1. Define node set V: specialized agent personas and MCP tools form individual vertices $v_i \in V$.
2. Evaluate pairwise capability similarity via Pearson correlation: $s_{ij} = \text{Cov}(\mathbf{x}_i, \mathbf{x}_j)/(\sigma_i \sigma_j)$.
3. Establish adjacency matrix $\mathbf{A}$: set $A_{ij} = 1$ when pairwise similarity meets threshold ($s_{ij} \geq \tau$), and $A_{ij} = 0$ otherwise.
4. Construct undirected communication edges E permitting message propagation exclusively between complementary pairings.

Step 3: Graph Attention Network (GAT) Initialization

1. Assign each vertex $v_i \in V$ initial embedding vector $\mathbf{h}_i = \mathbf{x}_i \in \mathbb{R}^d$ describing agent operational scope and tool signatures.
2. Instantiate weight matrix $\mathbf{W} \in \mathbb{R}^{d' \times d}$ and attention vector $\mathbf{a} \in \mathbb{R}^{2d'}$ via Xavier uniform initialization.
3. Configure network hyperparameters: learning rate η, multi-head count $K = 4$, and dropout probability p.

Step 4: Attention-Based Message Passing

1. Compute raw attention coefficients with connected neighbors $v_j \in \mathcal{N}(i)$:

$$e_{ij} = \mathrm{LeakyReLU}\Big(\mathbf{a}^T \big[\mathbf{W}\mathbf{h}_i \,\|\, \mathbf{W}\mathbf{h}_j\big]\Big)$$

2. Normalize coefficients via Softmax over $\mathcal{N}(i)$: $\alpha_{ij} = \exp(e_{ij}) / \sum_{k \in \mathcal{N}(i)} \exp(e_{ik})$.
3. Aggregate multi-head neighborhood features across $K = 4$ heads: $\mathbf{h}'_i = \|_{k=1}^{K} \sigma\Big(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^k \mathbf{W}^k \mathbf{h}_j\Big)$.
4. Multi-perspective collaborative attention heads capture:
   - Head 1: Intent & Query Semantics | Head 2: Model Capability & Context
   - Head 3: Swarm Persona & Tool Affinity | Head 4: Safety & Latency Budget

Step 5: Classification and Multi-Task Loss Computation

1. Route updated representations $\mathbf{h}'_i$ through dual specialized prediction heads:
   - Provider Routing Head: $\hat{\mathbf{y}}_{\text{route}} = \text{Softmax}(\mathbf{W}_{\text{route}}\mathbf{h}' + \mathbf{b}_{\text{route}})$
   - Agent Dispatch Head: $\hat{y}_{\text{agent},i} = \text{Sigmoid}(\mathbf{W}_{\text{agent}}\mathbf{h}'_i + b_i)$
2. Compute unified multi-task loss balancing routing accuracy with minimal swarm over-provisioning:

$$\mathcal{L}_{\text{class}} = \mathcal{L}_{\text{CE}}(\mathbf{y}_{\text{route}}, \hat{\mathbf{y}}_{\text{route}}) + \lambda \sum_{v \in V} \mathcal{L}_{\text{BCE}}(y_v, \hat{y}_v)$$

3. Optimize network parameters using AdamW with cosine learning rate decay and gradient clipping.

Step 6: Single-Stage Preference Alignment (ORPO)

1. Attach Low-Rank Adapters (LoRA) $\Delta\mathbf{W} = \frac{\alpha}{r}\mathbf{B}\mathbf{A}$ to base model attention projection layers.
2. Compute Supervised Fine-Tuning (SFT) loss on favored response y_w:
   $\mathcal{L}_{\text{SFT}} = -\frac{1}{|y_w|}\sum_{t=1}^{|y_w|} \log P(y_{w,t} \mid x, y_{w,<t})$.
3. Formulate Odds Ratio preference penalty: $\mathcal{L}_{\text{OR}} = -\log \sigma\big(\log[\text{odds}(y_w \mid x)/\text{odds}(y_l \mid x)]\big)$ where $\text{odds}(y \mid x) = \frac{P(y|x)}{1-P(y|x)}$.
4. Jointly optimize unified objective: $\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \beta \cdot \mathcal{L}_{\text{OR}}$ without separate reward models.

Step 7: Real-Time Runtime Inference and Swarm Execution

1. Ingest natural language prompt $\mathcal{T}_{\text{user}}$ via Next.js client interface and dispatch to FastAPI Hive Bridge.
2. Synthesize query feature vector: $\mathbf{x}_{\text{query}} = [\text{TF-IDF} \parallel \text{Dense Embeddings} \parallel \text{Context}]$.
3. Select optimal provider ($\hat{y}_{\text{provider}} = \text{argmax}(\hat{\mathbf{y}}_{\text{route}})$) and dispatch swarm ($\hat{\mathbf{y}}_{\text{agents}} = \mathbb{I}(\hat{\mathbf{y}}_{\text{dispatch}} \geq \tau_{\text{dispatch}})$).
4. Formulate task DAG and execute topological sort: $\pi = \text{TopologicalSort}(\mathcal{G}_{\text{active}})$.
5. Execute autonomous ReAct swarm loops querying Model Context Protocol (MCP) tool endpoints (Filesystem, Terminal, Browser, GitHub) until sub-tasks resolve.
6. Enforce calibrated safety gating: evaluate completion safety log-odds score against policy threshold τ_{safety} to intercept prompt injection.
7. Deliver real-time response: stream verified tokens live via Server-Sent Events (SSE) and commit conversation state locally to `PrismDB` (Dexie.js).

Step 8: Model Evaluation and Calibration

1. Benchmark trained routing and alignment models against holdout validation ($\mathcal{D}_{\text{val}}$) and test ($\mathcal{D}_{\text{test}}$) splits.
2. Compute comprehensive quantitative validation metrics:
   - Routing Accuracy & Macro-F1: Balanced intent and provider classification across rare classes.
   - Discrimination Capability (ROC-AUC / AUPRC): Safety gate discrimination under class imbalance.
   - Expected Calibration Error (ECE): $\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|$
   - Regression Precision (R^2 / MAE): Predictive fidelity for latency ($\hat{\ell}$) and monetary token cost ($\hat{c}$).

Step 9: Visualization, HUD Telemetry & Monitoring

1. Generate convergence diagnostics: plot training/validation loss trajectories and ROC frontiers.
2. Inspect attention heatmaps across $K = 4$ GAT heads to identify inter-agent collaboration bottlenecks.
3. Swarm Telemetry HUD: track live provider routing hit-rates, real-time token cost consumption ($\hat{c}$), per-agent latency profiling ($\hat{\ell}$), and safety policy pass-rates.

Thank You!

Questions & Discussion

Nobin Sijo | Pranav P | Prasanth P | Sankar Narayan S

Department of Computer Science & Engineering
College of Engineering, Karunagappally